Project 1

Detailed Data Analysis on Railway Gauges

Dataset Information:

I have used railway_gauges.csv dataset for this analysis. The file contains 4 kinds of attributes like Year, Broad Gauge, Narrow Gauge, Metre Gauge and Total. All the columns are of Integer type which had huge numbers as its data. Columns like Broad, Narrow and Metre Gauge along with Years are very prominent in this analysis. It contained 50 rows which is already cleaned and well structured.

Project Objective:

The main objective of this project is to ***"understand the trend between the usage of gauges"*** in railways. We are going to understand which gauge either narrow, broad or meter gauge is more prominently used in railway services. We are analyzing historical data and analyzing the patterns between them.

Modules Used:

The data analytics has been performed using pandas, NumPy and matplotlib libraries in order to understand the statistical behaviors of the data and understand the trend happening in between the data. Now, let's analyze the dataset. The below are the requirements:

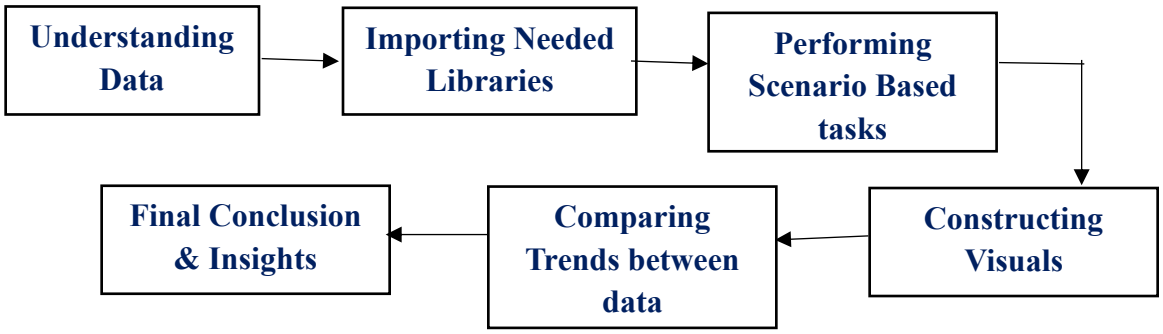
- ***IDLE*** – A compatible environment where we can run python files and see results
- ***Pandas*** – A python module used for data manipulation , filtering, aggregating and feature engineering tasks.
- ***NumPy*** – A Numerical Python which helps us to perform numerical calculations.
- ***Matplotlib*** – A visualization library helps to visualize data in bar, line, pie, and scatter.

Project Workflow:

The basic workflow of this project discusses about the below things.

1. Firstly, we are going to analyze and understand what the data actually saying. Here it just contains the 5 columns with 50 rows. It had three kinds of gauges values with their respective years. The data, details about the usage of gauges in several years.
2. Then, we can perform *historical data analysis* so that we can find the hidden trends and patterns.
3. We will perform the analysis in scenario based. Here we have 5 so we are going to find out the patterns and trends in various scenarios where we can understand it deeply.

4. To perform overall analysis, we import dataset and then do the following steps.



5. We perform analysis in the above flow in order to make a clear understanding on the analysis we do.

Steps for Constructing Analysis:

Step – 1:

Scenario – 1: Basic Data Loading and Cleaning:

Description: In scenario 1, I have imported libraries called NumPy, Pandas and Matplotlib. I have done basic data loading and cleaning operations on the dataset. This is the basic and important step in order to get accurate results. We are importing the dataset using pandas.

1.1 Loading the dataset into Pandas Data Frame:

Here we are importing the libraries and by using read_csv function from pandas we are importing the dataset as follows.

```
==========================================================================================
Scenario 1: Basic Data Loading & Cleaning
==========================================================================================
'''

#1.1 Load the dataset into a Pandas DataFrame.
data = pd.read_csv("C:/Users/Admin/Documents/pyCodes/data_Analytics/railway_gauges_project/railway_gauges.csv")
print("railway_gauges.csv.")
print("-------------------------------------------------------------")
```

So we will get output as

```
railway_gauges.csv.
-------------------------------------------------------------------
```

1.2 Display the first 5 rows and Column Name:

Now we will display the first 5 rows along with column name by using head() function. This function displays the default first five rows of the dataset in order to show the structure of the data. So the following is the snippet:

```
#1.2. Display the first 5 rows and column names.
print("Displaying first 5 rows of the dataset: ")
print(data.head())
print("-------------------------------------------------------------------")
```

So we have used head() function as above so we the results as below:

```
-----------------------------------------------------------------
Displaying first 5 rows of the dataset:
      Year  Broad Gauge  Metre Gauge  Narrow Gauge  Total
0  1964-65         3521         2891           464   6876
1  1965-66         3603         2913           470   6986
2  1966-67         3615         2911           481   7007
3  1967-68         3601         2920           506   7027
4  1968-69         3603         2921           508   7032
-----------------------------------------------------------------
```

1.3 Check for missing values and replace them with 0:

Now we have imported the data successfully, so we need to fill out the null values or NaN values in order to make sure the data is consistent for analysis. So we use fillna() function from pandas to replace null values or none values or NaN values with zero as below:

```
#1.3. Check for missing values and replace them with 0.
print("Checking for missing values and filling them: ")
updated_data = data.fillna(0)
#fillna() is used to fill the null values or NaN values existing in the dataset
```

As the dataset is already cleaned and well structured, there were no NaN values existing inside the data. But it checked whether values existed and made the data ready for analysis.

```
Checking for missing values and filling them:
Updated dataset is:
       Year  Broad Gauge  Metre Gauge  Narrow Gauge  Total
0   1964-65         3521         2891           464   6876
1   1965-66         3603         2913           470   6986
2   1966-67         3615         2911           481   7007
3   1967-68         3601         2920           506   7027
4   1968-69         3603         2921           508   7032
5   1969-70         3646         2913           499   7058
6   1970-71         3650         2915           501   7066
7   1971-72         3721         2876           496   7093
8   1972-73         3729         2871           498   7098
9   1973-74         3726         2874           479   7079
10  1974-75         3741         2873           471   7085
```

1.4 Convert all gauge columns (Broad, Metre, Narrow, Total) to numeric types:

Here we are going to convert all the columns into Numeric type so that while plotting them it will be easier for accessing. As data is already with Integer kind, only Year should be converted as it will be sometimes a string, we have implemented type conversion for all rows as below.

```
#1.4. Convert all gauge columns (Broad, Metre, Narrow, Total) to Numeric type
updated_data['Year'] = updated_data['Year'].astype(str).str.split('-').str[0].astype(int)
updated_data[['Broad Gauge','Metre Gauge','Narrow Gauge','Total']] = /
updated_data[['Broad Gauge','Metre Gauge','Narrow Gauge','Total']].astype(int)
print("Converted them into Numeric Type")
print("-----------------------------------------------------------------")
```

```
-------------------------------------
Converted them into Numeric Type
-------------------------------------
```

Step – 2:

Scenario – 2: Simple Visualization:

Description: We are constructing the basic visualizations like Line Plot just by using the existing columns. This visualization helps us to understand the basic pattern existing between the different labels.

2.1 Extract Year and Total Column

Here we are extracting Year and Total columns just by using pandas slicing method as below. It fetches the columns along with its respective rows.

```
#2.1. Extract Year and Total columns
df = updated_data[["Year","Total"]]
print(df)
```

The below will be the result:

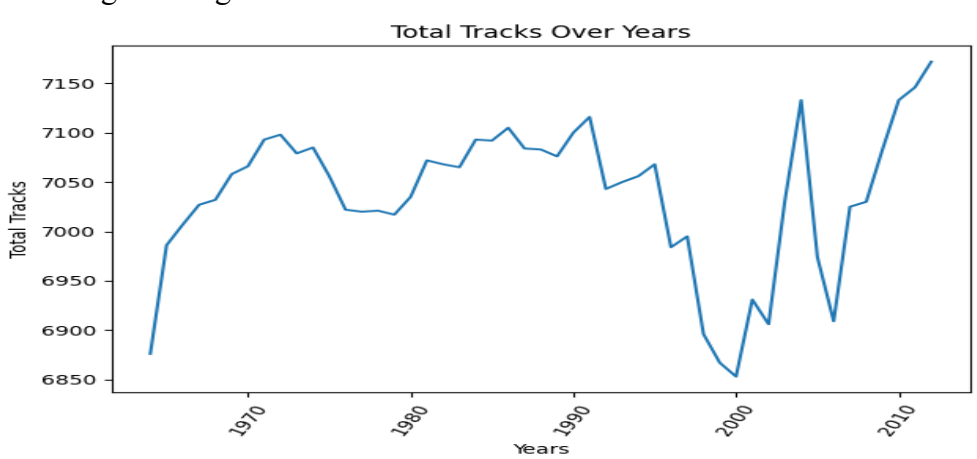
```
    Year  Total
0   1964   6876
1   1965   6986
2   1966   7007
3   1967   7027
4   1968   7032
5   1969   7058
6   1970   7066
```

2.2 Plot a Line graph showing Total tracks over years.

We are plotting a line graph for total tracks over years. We have used plot() function for plotting a line and title() for setting up a title for the chart, xlabel() and ylabel() are for setting up labels for x and y axis. We have used xticks for rotating the plot values to make it visible. tight_layout() is used for automatically setting up the subplots layouts.

```
#2.2. Plot a line graph showing Total Tracks over years
plt.plot(df['Year'], df['Total'])
plt.title("Total Tracks Over Years")
plt.xlabel("Years")
plt.ylabel("Total Tracks")
plt.xticks(rotation=60)
plt.tight_layout()
plt.show()
print("------------------------------------------------------------------")
```

Now we get the figure as:



2.3 Add Title, X and Y labels

Here we have added the title, x and y labels as below

```
plt.title("Total Tracks Over Years")
plt.xlabel("Years")
plt.ylabel("Total Tracks")
plt.xticks(rotation=60)
plt.tight_layout()
plt.show()
```

2.4 Identify whether the trend is increasing or decreasing

My observation from the plot is, the trend line is not constant, It has been booming and contraction phases since 1970 to 2010. But in the year 2010, it shows that it touched the booming period slightly above than previous years. So we can conclude that the demand for total tracks in increasing year by year.

Step – 3:

Scenario 3: Filtering and Bar Chart:

Description: We are filtering the data by several conditions and checking out the trends existing. We are going to construct a bar chart which explains about the distribution of three gauges and its dominance in years.

2.1 Filter the dataset for years after 2000:

We are filtering the dataset where Year will be after 2000. So it displays only data where years are above 2000. We are using pandas vectorization technique to fetch them.

```
#3.1. Filter the dataset for years after 2000.
filtered_df = updated_data[updated_data['Year'] > 2000]
print(filtered_df)
print("-----------------------------------------------------------------")
```

So the output looks like below:

```
-------------------------------------------------------
    Year  Broad Gauge  Metre Gauge  Narrow Gauge  Total
37  2001         5338         1310           283   6931
38  2002         5402         1220           284   6906
39  2003         5611         1190           230   7031
40  2004         5780         1134           219   7133
41  2005         5727         1018           229   6974
42  2006         5830          888           191   6909
43  2007         6000          806           219   7025
44  2008         6185          642           203   7030
45  2009         6320          543           220   7083
46  2010         6419          494           220   7133
47  2011         6494          432           220   7146
48  2012         6577          379           216   7172
-------------------------------------------------------
```

2.2 Select Broad Gauge, Metre Gauge and Narrow Gauge:

Now we are selecting the broad gauge, metre gauge and narrow gauge columns in order to plot them into a bar chart. So for selecting them we do the following.

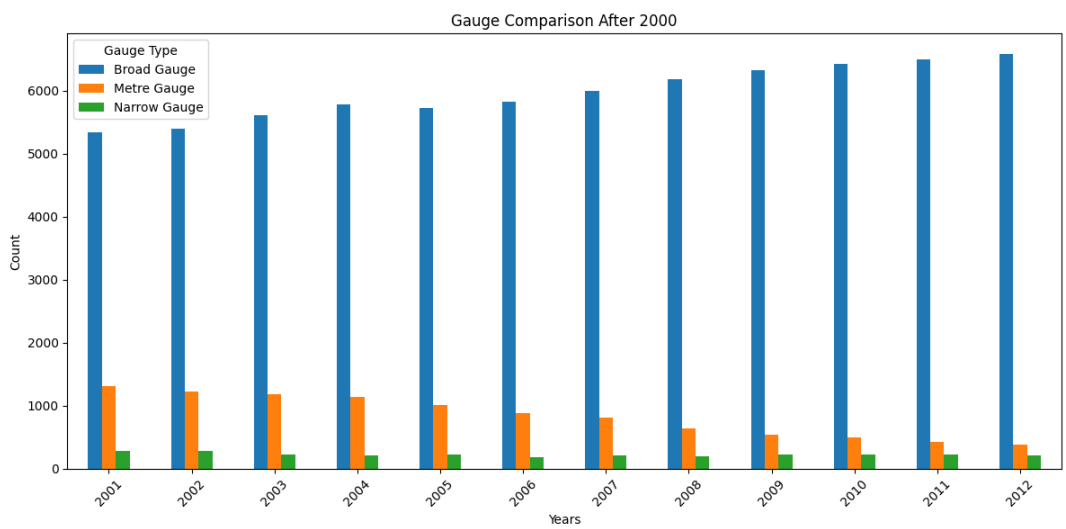
```
#3.2 Select Broad Gauge, Metre Gauge and Narrow Gauge
dataset = filtered_df[["Year", "Broad Gauge", "Metre Gauge", "Narrow Gauge"]]
```

2.3 Plot a grouped bar chart comparing all three gauges

So for plotting grouped bar chart, we are using the above dataset variable. Here we use plot() function only to plot the bar just by mentioning kind = 'bar' within the plot function as below. set_index is used for setting up Year on the X-axis.

```
#3.3 Plot a grouped bar chart comparing all three gauges
dataset.set_index('Year').plot(kind='bar', figsize=(12, 6))
plt.title("Gauge Comparison After 2000")
plt.xlabel("Years")
plt.ylabel("Count")
plt.xticks(rotation=45)
```

So the grouped bar chart of the three gauges looks like the below figure:



2.4 Add Legend and Proper Labels

So as we can see above graph, we have also defined the legend and labels for both x and y axis. Legend describes about the detailing of each bar, or simply represent bar's for what is it represented for.

```
#3.4 Adding legend and proper labels
plt.legend(title='Gauge Type')
plt.tight_layout()
plt.show()
```

2.5 Identify which gauge dominates in recent years.

Now we are calculating gauge dominants for several years just to know the cumulative demand of each gauge in respective years. The idxmax() will return the index label of the first occurrence of the maximum value along the specified axis.

```
#3.5 Calculate total for each gauge
totals = dataset[['Broad Gauge', 'Metre Gauge', 'Narrow Gauge']].sum()
print("Total: ", totals)
#Finding dominant gauge in recent years
high = totals.idxmax()
print(f"Most demanded Gauge in recent years: {high}")
```

So the above code gives the dominance of gauge in years as below:

```
-----------------------------------------------
Total:  Broad Gauge     71683
Metre Gauge     10056
Narrow Gauge     2734
dtype: int64
Most demanded Gauge in recent years: Broad Gauge
```

Step – 4:

Scenario 4: Feature Engineering and Pie Chart:

Description: In this scenario, we are analyzing the contribution of each gauge type and finding the Gauge which contributes more in Railway Services.

4.1 Calculate total sum of each gauge across all years.

We are calculating total sum of each gauge across all years so that we can find highest gauge which is more used gauge for railways.

```
#4.1. Calculate total sum of each gauge across all years

total_sum = updated_data[['Broad Gauge', 'Metre Gauge', 'Narrow Gauge']].sum()
print("Total sum of each gauge across all years is : ", total_sum)
```

So we get the output as below:

```
Total sum of each gauge across all years is :  Broad Gauge     224066
Metre Gauge     101798
Narrow Gauge     18953
dtype: int64
```

4.2 Create a new structure (Series/Data Frame) for totals.

Now we are converting data into Series for only totals.

```
#4.2 Create a new structure (Series/DF) for totals
summed_df = pd.Series(total_sum, name='Gauge_Totals')
print(summed_df)
```

So we get the result as below snippet:

```
Broad Gauge     224066
Metre Gauge     101798
Narrow Gauge     18953
Name: Gauge_Totals, dtype: int64
```

4.3 Plot a Pie Chart showing percentage contribution.

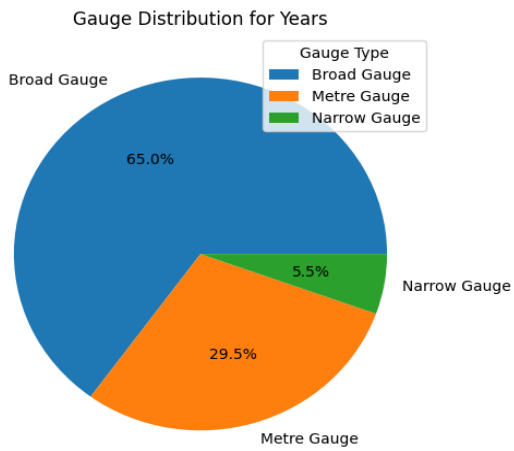
Now in this step we are plotting a pie chart showing percentage contribution of each gauge of total years. We can clearly understand the each gauge contribution so we can understand which gauge contributes more to railway services.

```
#4.3 Plot a pie chart showing percentage contribution
plt.pie(summed_df, labels=summed_df.index, autopct='%1.1f%%')
plt.title("Gauge Distribution for Years")
```

4.4 Add Percentage Labels (autopct).

```
#4.4 Add Percentage labels (autopct)
plt.legend(title="Gauge Type")
plt.tight_layout()
plt.show()
```

So after executing the above code for graph we get the following graph as below:



4.5 Interpret which gauge contributes the most.

Now, we can interpret that broad gauge is the most contributed kind for railway services when compared with other. Then after Broad gauge, the second contributed kind for railway services is Metre gauge.

```
Highest growth observed in 2003 with 1.81%
```

Step – 5:

Scenario 5: Advanced Analysis and Multiple Graphs:

Description: Here we are going to perform detailed analysis on finding out trend patterns from Three gauges.

5.1 Creating new columns (Broad_Gauge_%, Metre_Gauge_%, Narrow_Gauge_%)

Now we are creating calculated columns into a data frame for deeper analysis. We have calculated the percentage values by using the formula

Gauge% = Gauge_Value/Total_Gauge_Value * 100

```
#5.1 Create new Columns (Broad_Gauge%, Narrow_Gauge%, Metre_Gauge%)
updated_data['Broad_Gauge%'] = (updated_data['Broad Gauge'] / updated_data['Total']) * 100
updated_data['Metre_Gauge%'] = (updated_data['Metre Gauge'] / updated_data['Total']) * 100
updated_data['Narrow_Gauge%'] = (updated_data['Narrow Gauge'] / updated_data['Total']) * 100
```

5.2 Use NumPy (np.diff) to calculate yearly growth of Total Tracks

We use np.diff for calculating yearly growth of Total tracks as below:

```
#5.2 Use NumPy (np.diff) to calculate yearly growth of Total Tracks
diff = np.diff(updated_data['Total'])
prev_year_vals = updated_data['Total'].iloc[:-1].values
growth = (diff / prev_year_vals) * 100

updated_data['Yearly_Growth(in %)'] = np.insert(growth, 0, 0)
```

5.3 Plot Line graph for all gauges and Stacked Bar chart showing compositions

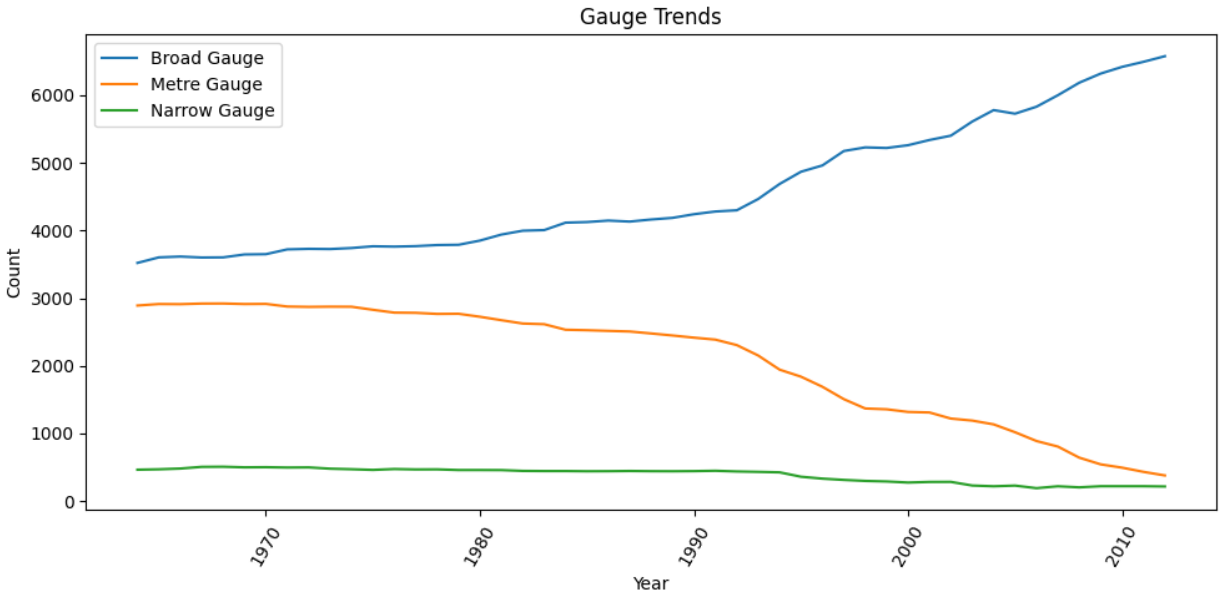
Now we use the below code for plotting line graph for all gauges and stacked bar chart showing compositions respectively.

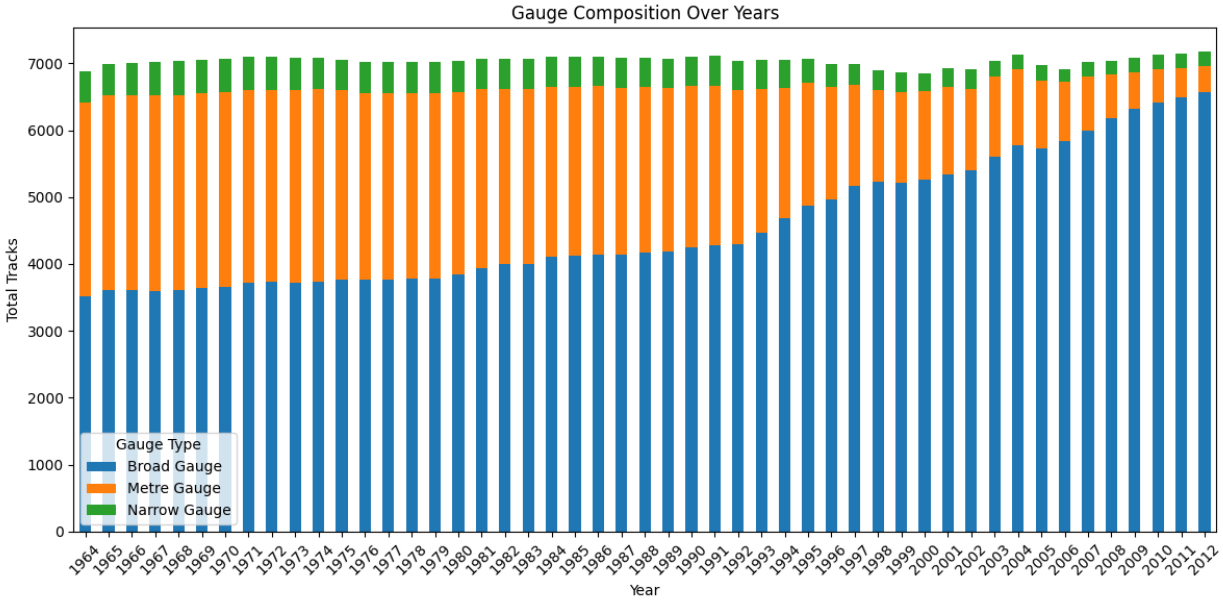
```
#5.3 Plot Line graph for all gauges and Stacked Bar chart showing comparison
plt.figure(figsize=(10, 5))
plt.plot(updated_data['Year'], updated_data['Broad Gauge'], label='Broad Gauge')
plt.plot(updated_data['Year'], updated_data['Metre Gauge'], label='Metre Gauge')
plt.plot(updated_data['Year'], updated_data['Narrow Gauge'], label='Narrow Gauge')

plt.title("Gauge Trends")
plt.xlabel("Year")
plt.ylabel("Count")
plt.legend()
plt.xticks(rotation=60)
plt.tight_layout()
plt.show()
```

```
updated_data.set_index('Year')[['Broad Gauge', 'Metre Gauge', 'Narrow Gauge']].plot(
    kind='bar', stacked=True, figsize=(12, 6)
)
plt.title("Gauge Composition Over Years")
plt.xlabel("Year")
plt.ylabel("Total Tracks")
plt.xticks(rotation=45)
plt.legend(title="Gauge Type")
plt.tight_layout()
plt.show()
```

So we get the following results for pie Stacked Bar plot and a line chart





5.4 Highlight Years with highest growth and Decline in any gauge

Here we are Highlighting the years with highest growth and Decline in any gauge

```python
#5.4 Highlight years with highest growth and Decline in any gauge

max_growth = updated_data['Yearly_Growth(in %)'].max()
year_max_growth = updated_data.loc[
    updated_data['Yearly_Growth(in %)'] == max_growth, 'Year'
].values[0]

print(f"Highest growth observed in {year_max_growth} with {max_growth:.2f}%")

decline = updated_data[
    (updated_data['Broad Gauge'].diff() < 0) |
    (updated_data['Metre Gauge'].diff() < 0) |
    (updated_data['Narrow Gauge'].diff() < 0)
]

print("Years with decline in any gauge:")
print(decline[['Year', 'Broad Gauge', 'Metre Gauge', 'Narrow Gauge']])
```

We get the output as below:

```
Years with decline in any gauge:
    Year  Broad Gauge  Metre Gauge  Narrow Gauge
2   1966         3615         2911           481
3   1967         3601         2920           506
5   1969         3646         2913           499
7   1971         3721         2876           496
8   1972         3729         2871           498
9   1973         3726         2874           479
10  1974         3741         2873           471
11  1975         3767         2828           461
12  1976         3762         2786           474
13  1977         3769         2783           468
14  1978         3785         2767           469
15  1979         3789         2769           459
16  1980         3851         2725           459
17  1981         3940         2674           458
18  1982         3997         2625           446
19  1983         4006         2615           444
20  1984         4116         2533           444
```

5.5 Provide Final conclusion:

Is the railway system shifting towards a single dominant gauge?

As per the analysis we have done, we can see that Broad Gauge is more used than the other two gauges in recent times. Narrow gauge is less dominant than any other gauge existing in the dataset. We can see railway system is shifting towards a single dominant gauge that is Broad Gauge

Conclusion:

The analysis of railway gauge data with multiple years gives us clear understanding about kinds of gauges, evolution and utilization of different gauge types. By viewing the dataset, I have observed that each gauge reached its peak usage in different years, but mostly there is drastic difference between broad gauge and narrow gauge. There is so much changes existing in infrastructure development over time.

The overall usage comparison revealed that Broad Gauge has the highest total value, making it the most dominant gauge type in the railway network. Trend analysis further supports this finding, as Broad Gauge shows a consistent increase over time while Metre Gauge is declining in pattern.

The visualizations, including bar charts and line plots, effectively highlight these patterns and provide a clear understanding of gauge distribution and trends. It helped me how a data driven approaches can be uncovered and can get meaningful insights about infrastructure development.

Scope of Developments:

As we have analyzed only historical or past data, it is only helpful for descriptive analysis and we got insights regarding past trends and patterns in railway gauge usage. In order to proceed with this analysis, it stood as a starter to get basic understanding of the dataset and its variables. But we can furtherly add predictive analysis module just by integrating additional datasets such as traffic, volume, route information, more comprehensive insights can be derived.

What I Have Learned ?

From this analysis, I learnt how to built a basic level analysis on dataset using NumPy, Pandas and Matplotlib. I learnt how pandas help us to clean, manipulate, and filter data using pandas vectorization concept. Then NumPy helped me to perform mathematical calculations like sum, mean and standard deviation etc. And Matplotlib is used for visualizing the data which can easily show the hidden patterns and trends existing in the data we have. Main thing is when we are performing analysis, it is purely depended on data quality and quantity. We can get highly accurate results when we have clean and well structured data, When data is insufficient and inconsistent, the outputs or patterns we get are less meaningful and will not add proper meaning to it.